

Interactive Visualization for Financial/Sales Data Using Python

Aim

To analyze financial or sales data and create interactive visualizations using Pandas and Plotly.

Procedure (Sequence of Steps)

Step 1: Install the required Python libraries such as pandas and plotly.

Step 2: Import the necessary libraries into the Python environment.

Step 3: Load the sales or financial dataset into a DataFrame.

Step 4: Display the dataset to understand its structure and contents.

Step 5: Examine the dataset information to identify data types and missing values.

Step 6: Convert the Date column into datetime format for time-based analysis.

Step 7: Clean the dataset by removing or handling missing values.

Step 8: Aggregate the data to compute total sales based on Date.

Step 9: Create an interactive line chart to visualize sales trends over time.

Step 10: Group the data by Region and calculate total sales.

Step 11: Create an interactive bar chart to compare region-wise sales.

Step 12: Generate a pie chart to visualize product-wise sales distribution.

Step 13: Create a scatter plot to analyze the relationship between Sales and Profit.

Step 14: Add interactive features such as color grouping for better comparison.

Step 15: Save the generated visualization as an HTML file for sharing.

Program (Python)

```
import pandas as pd

import plotly.express as px

# Load dataset

data = pd.read_csv("sales_data.csv")

# Display dataset
```

```
print(data.head())

print(data.info())

# Data preprocessing

data['Date'] = pd.to_datetime(data['Date'])

data = data.dropna()

# Aggregate sales by date

daily_sales = data.groupby('Date')['Sales'].sum().reset_index()

# Line chart for sales trend

fig1 = px.line(daily_sales, x='Date', y='Sales',
               title='Daily Sales Trend')

fig1.show()

# Bar chart for region-wise sales

region_sales = data.groupby('Region')['Sales'].sum().reset_index()

fig2 = px.bar(region_sales, x='Region', y='Sales',
              title='Sales by Region')

fig2.show()

# Pie chart for product distribution

fig3 = px.pie(data, names='Product', values='Sales',
              title='Product Sales Share')

fig3.show()

# Scatter plot for sales vs profit

fig4 = px.scatter(data, x='Sales', y='Profit',
                  color='Region',
                  title='Sales vs Profit Analysis')

fig4.show()
```

```
# Save visualization
```

```
fig1.write_html("sales_dashboard.html")
```

Output

Result

Interactive visualizations for financial/sales data were successfully created, enabling better analysis of trends, comparisons, and relationships for informed decision-making.

Exercise 10: Analysis of Geospatial Data for Urban Development Patterns

Aim

To analyze geospatial datasets and identify patterns in urban development using Python tools such as GeoPandas and Folium.

Procedure

Step 1: Install required libraries such as geopandas, folium, matplotlib, and pandas.

Step 2: Import necessary libraries into the Python environment.

Step 3: Load the geospatial dataset (e.g., shapefile or GeoJSON) into a GeoDataFrame.

Step 4: Display the dataset to understand spatial attributes and geometry.

Step 5: Check coordinate reference system (CRS) and ensure consistency.

Step 6: Clean the dataset by handling missing or invalid spatial data.

Step 7: Filter data based on urban attributes such as population, land use, or building density.

Step 8: Perform spatial aggregation to identify growth patterns across regions.

Step 8: Visualize spatial distribution using thematic maps (choropleth maps).

Step 9: Create interactive maps to explore urban development dynamically.

Step 10: Add markers or layers to represent key urban features (roads, buildings, zones).

Step 11: Analyze spatial relationships such as clustering or expansion trends.

Step 12: Interpret patterns such as high-density zones or urban sprawl.

Step 13: Export the generated maps as HTML files for sharing and reporting.

Program

```
import geopandas as gpd

import pandas as pd

import folium

import matplotlib.pyplot as plt

# Load geospatial dataset (GeoJSON or shapefile)

gdf = gpd.read_file("urban_data.geojson")

# Display data

print(gdf.head())

print(gdf.info())

# Check CRS

print(gdf.crs)

# Handle missing values

gdf = gdf.dropna()

# Plot basic map

gdf.plot()

plt.title("Urban Area Distribution")

plt.show()

# Create interactive map centered on mean location

map_center = [gdf.geometry.centroid.y.mean(), gdf.geometry.centroid.x.mean()]

m = folium.Map(location=map_center, zoom_start=10)

# Add GeoJSON layer

folium.GeoJson(gdf).add_to(m)
```

```
# Choropleth map (example: population density)

folium.Choropleth(

    geo_data=gdf,

    data=gdf,

    columns=[gdf.index, 'population'],

    key_on='feature.id',

    fill_color='YlOrRd',

    legend_name='Population Density'

).add_to(m)

# Add markers for important locations

for idx, row in gdf.iterrows():

    folium.Marker(

        location=[row.geometry.centroid.y, row.geometry.centroid.x],

        popup=str(row['area_name'])

    ).add_to(m)

# Save map

m.save("urban_development_map.html")
```

Output

Result

Successfully analyzed geospatial data and identified urban development patterns using Python. The visualization highlights spatial distribution, density variations, and growth trends, aiding urban planning and decision-making